

ENGLISH RESEARCH RELEASE / VERSION 1.0.0

DIKWP NeuroWeave Lab

The executable companion to
A Brief History of the Brain

Engineering and Research Handbook

Book authors: Yucong Duan and Zhongdao Wu

18 chapter-linked experiments

Reconstructive memory and state-pinned plans

Label-isolated evaluation and reproducible artifacts

Prepared 27 September 2026

Original English companion software and documentation. The named book authors are credited for the manuscript; author review or independent validation of this new implementation is not asserted.

Source code: MIT licence. Local Python execution. All bundled experimental evidence is synthetic. The manuscript and upstream archives are not redistributed.

Reader navigation

1. The central engineering contribution
 2. What readers can run immediately
 3. Architectural map
 4. Executing the book's minimal state model
 5. Two finite propositions made executable
 6. Numerical models and their equations
 7. Complete 18-chapter laboratory curriculum
 8. Memory, provenance and runtime API
 9. Evaluation and explicit denominators
 10. Verification and release integrity
 11. Practical extension
 12. Attribution and primary sources
- Appendix A. Bounded upstream source audit

Start with section 2 to run the system. Use chapter 10 for dependency-aware memory, chapter 13 for competing functional realizations, chapter 16 for the integrated state loop, and chapter 17 for the consent protocol.

1. The central engineering contribution

A useful companion should let readers do more than replay an attractive diagram. It should expose the difference between an observation and its interpretation, between a purpose and a fact, and between a proposed action and an authorised one. It should also give the reader a way to make an explanation fail. NeuroWeave turns these distinctions into objects, transitions and tests.

The core contribution is an integrated, executable workflow: receive a typed source, register its dependencies, derive a provisional conclusion, form a purpose-specific plan, bind that plan to the current evidence and resource state, obtain explicit local confirmation, execute only a synthetic operation, and record its declared synthetic feedback. When the evidence changes, the dependent conclusion becomes unusable until it is reconstructed. A previously confirmed plan is not allowed to survive a changed meaning of the state that justified it.

This is an engineering integration, not a claim that the component algorithms are newly invented. The numerical models are intentionally small enough to inspect. Their value is that they connect the book's questions to falsifiable statements, known equations, explicit data and reproducible software behavior. Scientific ambition is preserved by making its intermediate steps testable rather than by making stronger claims than the delivered artifacts support.

The targeted upstream assessment is deliberately uneven in scope because the evidence is uneven. Five relevant repositories were inspected as source trees containing code archives; their archive internals were not reviewed. PACT received a closer review of two complete source files. Its useful expanded tests and reproducibility structure are acknowledged. Specific confidence-leakage and denominator issues, rather than blanket judgments about the author's portfolio, motivate the new evaluation boundary.

2. What readers can run immediately

The project runs from its extracted directory using Python 3.11 or later. Its runtime, CLI, numerical engines, report generator and tests use the standard library. There is no required cloud service, API key, downloaded model, numerical accelerator or JavaScript build system. A dependency on a local Python interpreter remains; zero third-party runtime dependencies does not mean zero software requirements.

Open `outputs/reference/index.html` for a standalone report of the delivered reference runs. This page contains the actual recorded results, assumptions and interpretation limits. It does not call a server and does not recompute the models when opened. For interactive experiments run the local server:

```
python -m neuroweave serve --port 8765
```

Open `http://127.0.0.1:8765`. The sidebar contains the 18 chapter recipes. Each selection shows a testable prediction, a suggested perturbation, and a boundary on interpretation. The Run experiment button invokes the local Python engine. The browser displays selected scalar results, up to five time series, the complete JSON and the protocol/result hashes. All text returned from experiments is displayed as text rather than inserted as HTML.

Chapters 7, 10 and 15 accept specific numerical parameters in the browser. Other recipes intentionally expose fixed, readable reference configurations; deeper variation means editing a named Python model or adding a versioned protocol. A generic JSON editor is not advertised as arbitrary reprogramming of the runtime.

```
python -m neuroweave list
python -m neuroweave run chapter-10 --out outputs/my-memory-lab
python -m neuroweave demo --out outputs/my-demo
python -m unittest discover -s tests -v
python scripts/reproduce.py --out outputs/reproduced
```

The reproduction command runs the test suite before generating fresh results. It compares the 18 chapter JSON files and the benchmark JSON against the delivered reference. Comparison failures are visible and cause a nonzero process exit code. The HTML, test transcript and reproduction receipt are written beside the fresh JSON.

3. Architectural map

The architecture has four collaborating layers. First, typed records and numerical models produce finite observations and transformations. Second, reconstructive memory preserves source history and tracks which current conclusions remain usable. Third, the purpose/runtime layer decides whether a candidate is still justified by the exact state that was confirmed. Fourth, the experiment and reporting layer keeps predictions, truth scoring, provenance and evidence-domain limits separate.

The principal source modules are deliberately small and explicit:

- `evidence.py`: source kinds, intervals, context/time/unit comparison and claim-domain checks.
- `memory.py` and `ledger.py`: dependency-aware reconstruction, transactional history and replay.
- `purpose.py` and `runtime.py`: declared goals, local confirmation and state-pinned simulation.
- `models.py` and `evaluation.py`: finite mechanisms, label-isolated predictions and metrics.
- `labs.py`, `server.py`, `cli.py` and `reporting.py`: curriculum recipes and user-facing execution.

The dependency arrows in memory are not discovered from prose. A registered rule must name its source nodes. New nodes refer only to already-existing nodes, so insertion is topological and cyclic/self-referential additions are rejected. Computation uses a closed registry of sum, mean, scale and threshold operations. There is no `eval()`, imported expression execution, arbitrary file operation or remote action tool.

The evidence checker distinguishes four important situations. Incomplete means no usable bound or a required missing record. Inconsistent means supplied intervals do not overlap under the same declared context, unit and observation time. Imprecise means the compatible intersection is a non-singleton interval. Not comparable means the declared contexts, units or times differ. Interval intersection is a compatibility operation, not a statistical posterior or confidence interval.

4. Executing the book's minimal state model

Section 16.21 of the consulted manuscript separates a system state into six components:

$$X_t = (O_t, E_t, K_t, V_t, P_t, B_t)$$
$$X_{t+1} = F(X_t, Y_t, U_t, A_t, R_t)$$

O contains current source records and their histories. E contains explicit evidence relations. K contains derived, potentially reusable knowledge. V contains declared constraints and evaluation commitments. P contains the purpose and its revision. B contains the available resource conditions. Incoming observation Y, interpretation update U, candidate action A and consequence R are distinct participants in a transition. This is an operational teaching interpretation of the manuscript, not a discovered law of neural dynamics or an assertion that a brain has equivalent database fields.

ResearchRuntime.state() exposes these six partitions directly. A plan is pinned to a signature built from the current memory-log head, purpose contents and revision, and remaining resource budget. Confirmation and execution recompute that signature. New evidence, a changed goal, an intervening action or a resource change invalidates the old plan. The reader must reconstruct the relevant state and make a new plan.

The local execution produces a single-use receipt, debits a declared toy resource cost and appends explicitly synthetic feedback. The feedback is sampled around the planned value using a chosen seed; it is never represented as a real sensor observation. Only local simulated actions exist. Pending plans and consent are session-local, whereas the memory stream can be stored in SQLite and replayed after restart.

This design avoids treating DIKWP as merely a five-stage text template. Record, interpretation, relation, constraint and purpose participate in different operations and in different failure conditions. The implementation is one finite operationalisation; it does not redefine DIKWP's native semantics or prove that no other framework can represent the same distinctions.

5. Two finite propositions made executable

5.1 Observation alone can be insufficient

Let two task contexts share an observation o but require different actions a and b . Any deterministic observation-only function f must return the same value $f(o)$ in both contexts. Since a differs from b , that single value cannot be correct for both. Therefore, either the state must contain a context variable that distinguishes the tasks, or the system must acknowledge that its input is insufficient to select a unique action.

Chapter 5 constructs this witness with the same resource observation and two explicit tasks: preserve a specimen versus consume it for a test. A fixed observation-only policy returns the same action twice and succeeds once. The contextual policy uses the declared task field and returns the corresponding action. This does not demonstrate learned culture or novel reasoning. It demonstrates a representational necessity under the specified assumptions, just as the manuscript's finite proposition intends.

The point is not to proclaim the contextual implementation universally superior. A lookup table, a suitably designed state machine, or another framework can preserve the same context. The practical question is whether a real implementation keeps the particular context needed for its task, and whether that information remains available when goals, evidence or authorisation change.

5.2 Correction needs dependency information

Suppose the current text of a conclusion is identical in two histories. In the first, it depends on source A; in the second, it depends on unrelated source C. If A is withdrawn, the correct repair set differs. An algorithm that receives only the current text collection cannot necessarily distinguish those histories. Reliable selective repair therefore needs some retained dependency information or an independent process that can recover it with stated limitations.

The memory lab implements `sensor -> scaled -> above_threshold`, plus an unrelated source. The sensor starts at 6. The scale rule multiplies it by 2, and the threshold rule tests whether the result exceeds 10. A correction changes the sensor to 2. Both descendants become stale immediately; the unrelated source remains unchanged. Reading the threshold as current knowledge fails until recomputation runs. Recalculation then changes the scaled value from 12 to 4 and the threshold flag from 1 to 0.

Old observations are retained as historical events. The current interpretation is revised, not replaced by a fictitious story that the first reading never existed. Withdrawing the source leaves the descendants stale; recomputation cannot magically restore a missing evidential basis. A later explicit source revision can reactivate that source, after which permitted derivations can be reconstructed.

This finite engine only knows dependencies that were registered. It does not repair arbitrary natural-language beliefs automatically. That limit is valuable: it prevents an unknown provenance relationship from being falsely represented as verified knowledge.

6. Numerical models and what their equations mean

6.1 A checkable leaky integrate-and-fire model

The constant-current LIF model uses the usual passive-membrane equation, a threshold, a reset and a refractory duration. A primary teaching source for this model is Gerstner and colleagues' Neuronal Dynamics online text, section 1.3: <https://neuronaldynamics.epfl.ch/online/Ch1.S3.html>

```
tau * dV/dt = -(V - V_rest) + R*I
V_inf = V_rest + R*I
V(t+h) = V_inf + (V(t)-V_inf)*exp(-h/tau)
```

Here time is measured in ms, voltage in mV, current in nA and resistance in MOhm. The product MOhm times nA is mV. The implementation integrates the constant-input subthreshold segment analytically, locates threshold crossing within a recording interval, and advances through a refractory segment before continuing. The specified dt is a recording interval, not an Euler integration step.

For V_{inf} above threshold, the asymptotic interspike interval equals the refractory duration plus tau times the logarithm of $(V_{\text{inf}} - V_{\text{reset}}) / (V_{\text{inf}} - V_{\text{threshold}})$. At the default values, the first crossing is $20 \cdot \ln(4)$ ms; later intervals add 2 ms. Tests check the subthreshold analytic solution, first spike, subsequent interval and invariance across different recording intervals. A finite event budget prevents pathological parameter choices from producing unlimited simulated spikes.

The model does not describe ion-channel chemistry, spike waveform, dendritic structure or an entire brain. Those are not hidden features unlocked by the lab's chapter title.

6.2 Perception with a declared likelihood model

The cue experiment generates a balanced binary class z with values -1 and +1. Two conditionally independent Gaussian cues have means z and supplied standard deviations. With equal class priors, the log odds for the positive class are:

```
log_odds = 2*x_a/sigma_a^2 + 2*x_b/sigma_b^2
p_positive = 1 / (1 + exp(-log_odds))
```

The precision-weighted predictor implements this expression. Equal-precision and cue-a-only policies are explicit ablations. Each receives the same Observation fields; the ablations intentionally ignore some reliability or cue information. Truth labels belong to a different evaluator type and are not arguments to predict().

This is a correctly specified model check. Supplied reliabilities and Gaussian independence are strong assumptions that favour the matching formula. A good score shows that the pipeline and assumptions work together under the generator; it is not evidence that the system has learned unrestricted perception or outperformed real brains.

6.3 Observation versus intervention in a known world

The causal exercise supplies the structural model rather than claiming to discover it:

```
U, epsilon_X, epsilon_Y are independent standard normal variables.
X = U + epsilon_X
Y = beta*X + gamma*U + epsilon_Y
```

The population observational regression slope of Y on X is $\beta + \gamma/2$ because $\text{Var}(U) = \text{Var}(\epsilon_X) = 1$. Replacing the equation for X with $\text{do}(X=1)$, versus $\text{do}(X=0)$, while holding the exogenous variables fixed, yields a paired difference of β . At the defaults $\beta=1$ and $\gamma=2$, the observational slope approaches 2 while the paired intervention effect is 1. Setting γ to zero removes this confounding path.

The exact paired effect follows the additive equations and common exogenous noise. It is not an independently discovered causal effect, an identification algorithm for unknown graphs, or a biomedical inference from observational patient records.

6.4 Matched behavior and targeted functional tests

Two hand-constructed dynamical realizations implement the same transfer function: a persistent recurrent workspace and a persistent sensory filter. Under the default baseline, their output sequences match. An intervention labelled `workspace_lesion` removes recurrence in the first realization but leaves the sensory-filter state update unchanged. Their observable responses then diverge.

A `report_lesion` zeros the output while preserving the latent proxy, showing within this model why silence alone does not uniquely determine internal state. Sham handling leaves the mechanism unchanged. Rescue restores original parameters in a new paired trial; it is not a demonstration of biological recovery or within-trial tissue repair. These controls teach functional identifiability and explicit intervention semantics. They do not decide whether either realization has subjective experience.

6.5 Resource, propagation and adaptation exercises

The resource controller compares always-work behavior with a policy that recovers when its remaining budget is low. It separately records completed work, final resource, inflow, actual expenditure and the balance residual. There is no unqualified success score and no claim to model sleep, metabolism or thermodynamic entropy.

The propagation exercise compares a finite chain with a directed cycle under the same edge gain. A lesion disables transfer through one declared node. This small graph illustrates dependence on topology, not a ranking of species or a biological connectome.

The adaptation exercise compares a cumulative mean with an exponential moving average under a known distribution reversal. Errors are computed before each update. Faster adaptation has a stability cost; removing the reversal is a useful counter-condition. Neither estimator is presented as a complete developmental or synaptic learning theory.

7. The complete book-to-laboratory curriculum

Each chapter card below names its implemented engine, prediction, executable command, and a perturbation for readers. Several cards reuse the same engine for a different question. These are teaching interpretations of the identified manuscript, not a claim that each chapter has a complete physiological implementation.

7.1 Chapter 01: Three Histories: Life, Mind, and Self-Discovery

Laboratory: Evidence Is Not Its Interpretation. **Engine:** evidence.

Prediction: Typed records prevent synthetic results from silently becoming biological validation.

```
python -m neuroweave run chapter-01 --seed 17
```

Inspect the distinct incomplete, imprecise and inconsistent outputs. Then inspect `biological_claim` and `experience_claim`: provenance organisation can reveal an unsupported promotion of evidence without claiming to settle the underlying scientific question. Suggested extension: add a record with a different observation time and show why it cannot be silently merged.

Perturbation: Remove the source kind and ask which claim changes without new measurement.

Interpretation boundary: Record provenance is not source authentication or scientific peer review.

7.2 Chapter 02: Before Neurons: How Life Maintains Itself

Laboratory: A Body Has a Budget. **Engine:** homeostasis.

Prediction: A resource-aware controller sustains more completed work than always-work in the specified toy environment.

```
python -m neuroweave run chapter-02 --seed 17
```

Compare useful completed work, not only terminal resource. The always-work policy can consume its initial budget rapidly and then remain unable to complete work. The resource-aware policy accepts recovery steps. Check that final resource equals initial resource plus inflow minus actual expenditure to numerical tolerance.

Perturbation: Increase resource inflow or work cost and compare energy balance and useful work.

Interpretation boundary: Arbitrary resource units are not metabolism, entropy, health advice, or evidence of life.

7.3 Chapter 03: The Origin of Nervous Systems: From Local Response to Coordination

Laboratory: Local Signals, Global Coordination. **Engine:** network.

Prediction: Recurrence prolongs activity relative to a finite chain at the same edge gain.

```
python -m neuroweave run chapter-03 --seed 17
```

Observe the total activity trajectory in both graph topologies. A chain eventually loses its initial signal because no edge returns it to an earlier node. A cycle can preserve a decaying circulation. Its persistence is a property of the specified edge dynamics, not an assertion of awareness or biological coordination.

Perturbation: Set the gain to zero in a copy and inspect loss of propagation.

Interpretation boundary: Graph recurrence alone does not explain the evolutionary origin of neurons.

7.4 Chapter 04: Diverse Brains: Comparative Evolution and Organizational Possibility

Laboratory: Different Architectures, Different Failure Paths. **Engine:** network.

Prediction: A lesion changes propagation differently in a chain and cycle.

```
python -m neuroweave run chapter-04 --seed 17
```

Read the intact and lesioned structures together. A visible change in topology can alter when activity disappears, but a particular output difference does not identify every possible internal mechanism. Modify the disabled target in `models.network()` and keep the original as a named comparison instead of silently replacing the reference.

Perturbation: Move the disabled edge in code; distinguish architecture from organism rankings.

Interpretation boundary: Six-node graphs are neither connectomes nor a ladder of species.

7.5 Chapter 05: The Emergence of Humans: Tools, Cooperation, and Cumulative Culture

Laboratory: The Same Observation, Two Tasks. **Engine:** context.

Prediction: An observation-only deterministic policy cannot satisfy two contrary task requirements.

```
python -m neuroweave run chapter-05 --seed 17
```

The two records deliberately have the same observation string. The correct actions differ only because the task conditions differ. The observation-only witness gets one of two correct. This is a finite proof example, not an empirical claim that all non-DIKWP architectures fail contextual tasks.

Perturbation: Exchange task labels without changing the observation; inspect the selected action.

Interpretation boundary: The context labels are supplied, not learned cultural understanding.

7.6 Chapter 06: The Brain Across a Lifetime: Development, Learning, and Plasticity

Laboratory: Learning Under a Changing World. **Engine:** plasticity.

Prediction: An exponential learner adapts more rapidly to the specified reversal than a cumulative average.

```
python -m neuroweave run chapter-06 --seed 17
```

The default world reverses midway through the run. Read `post_change_prediction_mse` alongside the trajectory, and verify that predictions are scored before the next observation is used for an update. A

stable-world variant can expose the noise sensitivity of the faster estimator; such a variant should have its own protocol name.

Perturbation: Remove the reversal and compare the cost of fast adaptation.

Interpretation boundary: Two scalar estimators are not a developmental or synaptic plasticity theory.

7.7 Chapter 07: Electricity and Chemistry: The Working Language of Nervous Systems

Laboratory: A Neuron Model You Can Check. **Engine:** lif.

Prediction: Constant-current spike intervals agree with the analytic LIF expression.

```
python -m neuroweave run chapter-07 --seed 17
```

Run `current_na=0.1` for a subthreshold example and `current_na=0.2` for a spiking example. Compare `dt_ms=0.3` with `dt_ms=0.8`: recording density changes while the analytic crossing times remain equal up to floating-point tolerance. This is stronger verification than merely plotting an oscillating line.

Perturbation: Vary current and recording interval; test subthreshold and suprathreshold cases.

Interpretation boundary: The model does not generate action-potential shape, ion-channel chemistry, or a whole brain.

7.8 Chapter 08: Brain and Body: Metabolism, Immunity, Interoception, and Sleep

Laboratory: Rest as a Resource Decision. **Engine:** homeostasis.

Prediction: A controller can preserve its ability to act by accepting short-term rest.

```
python -m neuroweave run chapter-08 --seed 17
```

Reuse the resource engine to ask a different question: when can rest preserve future capacity to act? Compare the path of the budget with cumulative useful work. The numerical result is not a prescription about human rest, sleep duration or illness. It is an explicit demonstration of a resource-constrained policy trade-off.

Perturbation: Compare cumulative work with final resources; neither alone defines success.

Interpretation boundary: This is not a physiological model of sleep, immunity, or interoception.

7.9 Chapter 09: Perceiving the World: Evidence, Attention, and Active Construction

Laboratory: Reliable Cues, Calibrated Beliefs. **Engine:** perception.

Prediction: Known cue precision improves Brier loss under the specified Gaussian generator.

```
python -m neuroweave run chapter-09 --seed 17
```

Inspect the Observation examples and verify that labels are absent. The full label set belongs to scoring. The separate 20-seed benchmark alternates which cue is reliable, so the one-cue ablation cannot always rely on the same favourable channel. The known reliabilities remain a declared modelling advantage.

Perturbation: Swap cue reliabilities while keeping the prediction interface label-free.

Interpretation boundary: A correctly specified Gaussian model is an assumption, not evidence of universal perception.

7.10 Chapter 10: The History of Memory: Retention, Reconstruction, and the Future

Laboratory: Correct the Past Without Inventing It. **Engine:** memory.

Prediction: A source revision invalidates exactly its dependency descendants and preserves unrelated nodes.

```
python -m neuroweave run chapter-10 --seed 17
```

Run the default correction from 6 to 2, then use `corrected_source=8`. The same dependency mechanism should compute a different resulting flag because the new evidence differs. A successful repair process is not defined as always lowering confidence or always rejecting a conclusion; it applies the registered rule to the current supported state.

Perturbation: Retract the source after recomputation and attempt to read its descendant.

Interpretation boundary: Dependencies are explicitly registered; the system does not infer them from arbitrary text.

7.11 Chapter 11: The Direction of Action: Emotion, Value, and Purpose

Laboratory: Change the Goal, Keep the Facts. **Engine:** purpose.

Prediction: A purpose reversal can change actions while the source-state hash stays fixed.

```
python -m neuroweave run chapter-11 --seed 17
```

Compare the source-state hashes before and after switching from explore to preserve. They remain identical while the toy action differs. Then inspect `zero_budget_action`. Resource limits and utility priorities can change behavior without rewriting the observation into a more convenient fact.

Perturbation: Change only the resource budget and inspect action versus evidence state.

Interpretation boundary: Utility settings are declared engineering choices, not a measure of moral or experiential value.

7.12 Chapter 12: Language and Social Minds: How Meaning Is Jointly Built

Laboratory: Meaning Has Conditions and Owners. **Engine:** semantics.

Prediction: Structured negation and owner fields can be preserved or detectably lost in a transfer.

```
python -m neuroweave run chapter-12 --seed 17
```

The communication record carries owner, action, negation and context. Its lossy transfer drops negation and is marked unfaithful. The checker verifies these four schema fields only. It does not parse free-form language or infer that a person's fluently predicted sentence has been authorised by that person.

Perturbation: Drop the negation field and inspect the semantic-difference report.

Interpretation boundary: This is schema-level checking, not an automatic natural-language understanding engine.

7.13 Chapter 13: The Boundaries of Consciousness: Experience, Report, and Testable Theories

Laboratory: Matched Behavior, Different Mechanisms. **Engine:** mechanisms.

Prediction: Observationally matched realizations diverge under a targeted internal lesion.

```
python -m neuroweave run chapter-13 --seed 17
```

Inspect every condition for both realizations, not only the row with the largest effect. Baseline matching makes the intervention informative under the supplied construction. Report-channel loss and recurrence loss have different internal consequences. The appropriate conclusion is about these functional mappings, not subjective experience.

Perturbation: Compare sham, workspace lesion, report lesion, and parameter-restored rescue.

Interpretation boundary: Functional discrimination is not a test proving subjective experience.

7.14 Chapter 14: When Maintenance Fails: Injury, Infection, Degeneration, and Mental Disorders

Laboratory: Loss of Output Is Not Loss of Every State. **Engine:** network.

Prediction: Disabling an output channel can eliminate reports while leaving a latent proxy unchanged.

```
python -m neuroweave run chapter-14 --seed 17
```

The output-loss example has report energy zero and a nonzero latent proxy. This prevents one particular modelling error: equating absent output with absence of every internal variable. It supplies no clinical diagnostic rule and cannot be used to classify an unresponsive patient or a real artificial system.

Perturbation: Compare internal lesion and output lesion before interpreting silence.

Interpretation boundary: This toy cannot diagnose consciousness, neurological injury, or any medical condition.

7.15 Chapter 15: Seeing, Mapping, and Intervening: From Brain Maps to Causal Models

Laboratory: Observation Is Not Intervention. **Engine:** causal.

Prediction: The observational slope differs from the paired do-effect in the confounded structural model.

```
python -m neuroweave run chapter-15 --seed 17
```

Use beta and gamma to separate the structural effect from the confounding contribution. Increasing n reduces sampling fluctuation in the observational slope; it does not remove confounding. The paired do-effect is exact under this additive synthetic construction, not an achievement of recovering an unknown causal graph.

Perturbation: Set the confounder coefficient gamma to zero and repeat.

Interpretation boundary: The graph and equations are supplied. Causality is not automatically identified from observational data.

7.16 Chapter 16: From Brain to AI: Semantics, World Models, and Reconstructive Memory

Laboratory: The Book's State Model, Executed. **Engine:** integrated.

Prediction: Purpose changes preserve sources; source revisions trigger dependency repair and changed candidate actions.

```
python -m neuroweave run chapter-16 --seed 17
```

This is the integrated demonstration. It proposes a plan, changes purpose and rejects the old plan; proposes again, corrects evidence and rejects that plan; reconstructs the dependency graph, creates and confirms a fresh plan, then records synthetic feedback and a resource debit. Inspect each O/E/K/V/P/B snapshot rather than only the final receipt.

Perturbation: Change exactly one of evidence, purpose, consent, or resources; compare state partitions.

Interpretation boundary: This is one finite realization of the book model, not its unique implementation or a brain equation.

7.17 Chapter 17: Brain-Computer Interfaces: Restoring Channels, Not Speaking for People

Laboratory: Prediction, Confirmation, Execution. **Engine:** consent.

Prediction: An unconfirmed, revoked, expired, or changed proposal cannot produce even a local execution receipt.

```
python -m neuroweave run chapter-17 --seed 17
```

The demonstration rejects five cases: missing confirmation, changed message, reuse after execution, revocation and expiration. It then exhibits a valid local receipt with `external_side_effect=false`. Owner labels are declarations inside the process, so this is a consent-protocol teaching model rather than a real identity system.

Perturbation: Confirm one payload and attempt to execute a modified message.

Interpretation boundary: Actor names are simulated identities; no real BCI decoder, authenticated person, or actuator is connected.

7.18 Chapter 18: An Unfinished History: The Open Future of Life, Semantics, and the Brain

Laboratory: An Open Future Needs Revisable Claims. **Engine:** closure.

Prediction: An experiment can record a falsifier and preserve history when its supporting source is withdrawn.

```
python -m neuroweave run chapter-18 --seed 17
```

Withdraw the source and inspect the reopened claim and stale descendants. The unrelated source remains usable. A withdrawal is not deletion: the log still records how the old conclusion arose. The exercise ends with revisable evidence and a preserved audit trail, not with a claim that the whole scientific question has been closed.

Perturbation: Withdraw the source and verify that an unrelated conclusion is retained.

Interpretation boundary: A complete software loop is not proof of complete scientific explanation.

8. Memory, provenance and runtime API guide

8.1 Registering a source and a derivation

The source API records a numeric value, a source identifier, a context, a unit and a kind. A source kind describes the declared evidence role; it is not a certificate that an external webpage, sensor or laboratory is authentic. Contexts and units must be compatible for the finite derivation rules. Cross-context mappings and unit conversions must be performed by separately reviewed code rather than hidden inside a sum.

```
from neuroweave.ledger import Ledger
from neuroweave.memory import Memory

log = Ledger("my-memory.sqlite")
memory = Memory(log)
memory.add_source("a", 6, source="synthetic://sensor", context="trial")
memory.derive("b", ["a"], {"op": "scale", "factor": 2}, context="trial")
changed = memory.revise("a", 2, reason="Calibration correction")
memory.recompute()
assert memory.read("b") == 4
checkpoint = log.verify()
log.close()
```

This creation example expects a new database. Reusing its IDs in the same persisted database is intentionally rejected; reload the existing state and revise it instead. A source revision invalidates its transitive descendants. Recompute visits nodes in the insertion/topological order and only reactivates a stale derivation whose parents are all active. A retracted node remains retracted. Historical source values stay in the append-only event stream.

8.2 Replay and concurrency

Each log entry stores its sequence, kind, payload, previous hash and content hash. SQLite transactions make an individual append atomic. An expected-head check rejects a write made from a stale local Memory view. Construct a fresh Memory from the log before intentionally retrying a conflicting write; do not simply overwrite the newer state. Use the runtime as a single-writer teaching process, not as a multitenant service.

Replay verifies the chain and reapplies registered memory operations. Current source and derived values should then match the original snapshot exactly. A separately retained checkpoint is required to detect an otherwise-consistent deleted suffix. The code cannot detect a privileged replacement of both log and checkpoint. Hashes also cannot show that the original observation was true.

8.3 Planning against a particular state

The unified runtime binds a local candidate to the current evidence state. The example below uses the predefined synthetic memory factory; production input adapters are not part of the release.

```
from neuroweave.labs import make_memory
from neuroweave.purpose import Purpose
from neuroweave.runtime import ResearchRuntime

memory = make_memory()
runtime = ResearchRuntime(memory, Purpose("researcher", "trial", "explore", 2))
```

```
runtime.propose("candidate-1", "above_threshold", now=0, cost=0.5)
runtime.confirm("candidate-1", actor="researcher", now=1)
result = runtime.execute("candidate-1", actor="researcher", now=2, seed=17)
assert result["receipt"]["external_side_effect"] is False
assert result["resource_remaining"] == 1.5
memory.ledger.close()
```

A purpose change cannot silently raise the remaining budget, and changing the owner requires a new runtime. A source correction between proposal and confirmation makes the plan stale. Another execution changes the log and resources, invalidating other pending plans. Modifying a returned plan dictionary does not modify the internal copy. These are useful bounded engineering guarantees, not authenticated human authority.

Resource debit, session receipt and feedback persistence are not one distributed transaction. A failure while writing feedback requires inspection rather than blind retry. Only synthetic local actions are present, so the runtime does not promise rollback of real-world consequences it cannot control.

9. Evaluation that cannot win by hiding its denominator

The benchmark separates three types: Observation, Prediction and Truth. Observation has no answer label. Prediction is a probability associated with a case ID. Truth is used only by the evaluator. `predict()` accepts the observation type and a known model name. Tests deliberately change the truths and establish that probabilities remain unchanged. This prevents the particular inspected upstream confidence-leakage pattern within the supplied implementation; it does not sandbox hostile code in the process.

Let N be the number of declared cases and M the number of returned predictions. Coverage is M/N . All-case accuracy divides correct predictions by N , so an omitted case receives no credit. Observed-only accuracy divides by M and is reported separately, or as null when M is zero. Unknown case IDs and duplicate IDs are rejected rather than discarded, overwritten or opportunistically matched to more favourable labels.

For a returned probability p and binary label y , Brier loss is $(p-y)^2$. A missing prediction receives the maximum binary loss of 1. The mean uses all N cases. Calibration bins report their returned-case counts; they do not disguise missing outputs as good calibration. No universal weighted semantic-quality or consciousness score is produced.

The reference uses 20 seeds, 240 cases per seed and three explicit predictors. It constructs and hashes the protocol before generating cases, computes predictions before calling the scorer, and preserves prediction hashes. This is a local commitment to configuration, not an independently time-stamped preregistration. The seeds are public and the generator is known. There was no model training or blinded external study.

Confidence intervals use 1,500 bootstrap resamples of the 20 seed-level Brier means. The paired comparison uses the 20 within-seed precision-minus-equal differences. Seed-level resampling avoids treating a dense time series as many independent trials. It still represents uncertainty only under the supplied synthetic generator, not the broader uncertainty of human cognition or real deployment.

The protocol's directional criterion is that precision weighting has lower mean Brier loss than equal weighting. This is expected under the correctly specified likelihood. The report includes the criterion, actual effect, interval and limitations. It does not select a new metric after seeing the data or turn a model-check result into a claim of universal superiority over other repositories.

10. Verification, release integrity and actual execution status

The unit and integration tests examine numeric bounds, nonfinite inputs, evidence comparability, exact dependency invalidation, old-source retention, reconstruction, replay, stale-writer rejection, log tampering, consent expiry/revocation/reuse, state signature invalidation, label isolation, denominator accounting, numerical solutions, chapter determinism, CLI operation and HTTP input controls.

The authoritative test count and actual runtime environment are recorded in `validation/TEST_RECEIPT.json`. The transcript is included so that a reported pass can be traced to named tests. This is stronger than an unexecuted checklist, while still being limited to the supplied implementation and cases.

The reference experiment artifacts are regenerated after implementation changes. A second run is compared against all 19 reference JSON files. Browser tests, when recorded in `validation/UI_RECEIPT.json`, are separate from Python tests and name the actual checks performed. A screenshot is a usability aid, not evidence of numerical correctness. The supplied GitHub Actions configuration is a future repository check; it was not remotely executed just because a workflow file exists.

SHA256SUMS lists the shipped files, excluding the checksum file itself. Running `scripts/verify_release.py` checks the delivered bytes. A checksum list provided beside a file does not authenticate its author or prove no one replaced both. An eventual publisher should add a trusted signed release or independent archive if authenticity is part of the distribution requirement.

11. Practical extension without overstating integration

A new scientific model should supply a named mechanism, explicit units, a bounded input contract, reproducible parameters, a competing or ablated implementation, and an interpretation boundary. Reusing the chapter UI is straightforward, but merely adding a menu entry is not a delivered scientific capability. A new reference should fail when a relevant mechanism is removed and should also include a condition where the expected advantage is absent or reduced.

A text or model-provider adapter should return candidate structured records with sources and declared uncertainty. It should never convert generated prose directly into a permission grant or an empirically validated biological claim. The source of truth for benchmark labels must remain outside the prediction interface. Network credentials and provider code belong in separately reviewed optional integrations; none is silently required by this release.

A deployment handling real human data requires consent that is authenticated to a real person, lawful collection, privacy controls, retention/deletion behavior and appropriate independent review. A deployment controlling real devices also needs trusted time, durable authorisation, bounded actuators, failure containment and actual recovery guarantees. These are missing deployment capabilities, not restrictions added to the MIT licence or claims that the research software cannot be extended.

A legitimate next contribution can be small and important: improve evidence-group validation, add a tested contextual adapter, introduce efficient reverse dependency indexes, or compare models on independently obtained data. No calendar, budget, collaborator commitment or deployment claim is invented by this handbook.

12. Attribution and sources

The primary book anchor is the user-provided manuscript A Brief History of the Brain, by Yucong Duan and Zhongdao Wu, complete version dated 27 September 2026. The teaching state model and two finite propositions are associated with section 16.21; reconstructive memory also connects to chapter 10. Functional experimental controls connect to chapter 13, causal interpretation to chapter 15, and the distinction between prediction, confirmation and execution to chapter 17. The original manuscript is not included.

The conventional LIF model source is the Neuronal Dynamics online textbook, section 1.3:
<https://neurondynamics.epfl.ch/online/Ch1.S3.html>

Python's standard-library HTTP implementation is documented at:
<https://docs.python.org/3/library/http.server.html>

The precise upstream repository/source evidence, tree/blob identifiers and limits of the review are reproduced in the following appendix. All links are references, not runtime dependencies. No novel ownership claim is made over conventional numerical methods, general provenance principles or explicit consent state machines.

Appendix A. Bounded upstream source audit

Inspection date: 27 September 2026. **Owner:** YucongDuan. **Purpose:** identify actionable obstacles to an executable companion for *A Brief History of the Brain*, not score a person's research programme or claim an exhaustive assessment of all their repositories.

What was actually inspected

Six relevant public repositories were selected. Five were inspected at the recursive Git-tree level, with available README context. PACT was inspected at repository, source-directory, and two complete Python-source-file levels. GitHub's connected repository interface returned the trees and source text. The five archive contents were not unpacked or executed in this assessment. Network restrictions prevented local clones and binary archive retrieval. These restrictions are an audit limit, not evidence that the archives are empty or unusable.

The identifiers below distinguish **tree objects** from **file blobs**. They are not mislabelled as commit IDs. A future audit should resolve the corresponding commits and review archive internals before making broader implementation comparisons.

1. OmniMemory-OS — repository packaging / documentation scope

Repository: <https://github.com/YucongDuan/OmniMemory-OS>

Observed recursive tree: 8811996b9e845ef214f3036330af5907cf1d874f. The untruncated tree contains LICENSE, README.md, and MESH92-OmniMemory-OS-2026-v1.0.0.zip (1,452,883 bytes).

Evidence endpoint: <https://api.github.com/repos/YucongDuan/OmniMemory-OS/git/trees/8811996b9e845ef214f3036330af5907cf1d874f>

The available README describes purpose-aware memory capsules, provider-bound budgeting, separate planning/sending, and source-deletion/tombstone propagation. Those are substantive ideas to retain. This audit does not claim that reconstructive or deletion-aware memory is absent. The directly observed limitation is that the implementation is not expanded in the inspected Git tree, making code navigation, line-level review, and ordinary source-diff workflows less immediate.

Response in NeuroWeave: expanded source; a small inspectable dependency engine; selective invalidation; explicit recomputation; source-history replay; regression tests. This is a tested companion-specific implementation, not a verified replacement for all OmniMemory provider adapters or features.

2. WorldWeave — repository tree scope

Repository: <https://github.com/YucongDuan/WorldWeave>

Observed tree: 0c6ff7a6e3441ae62ff59ef20c98ad77b11def44. It contains LICENSE, README.md, and worldweave_system_v0_9.zip (62,838 bytes).

Evidence endpoint: <https://api.github.com/repos/YucongDuan/WorldWeave/git/trees/0c6ff7a6e3441ae62ff59ef20c98ad77b11def44>

Only the delivery structure is established here. No claim is made about numerical solvers, tests, world-model quality, or license details inside the archive.

Response: directly inspectable finite world models, an explicit observation interface, intervention controls, and generated result artifacts. No upstream WorldWeave code was run in a head-to-head experiment.

3. DIKWP-NeuroBody-SemanticOS-Platform-Package — tree scope

Repository: <https://github.com/YucongDuan/DIKWP-NeuroBody-SemanticOS-Platform-Package>

Observed tree: cc303795892906a331f7bad2d4f4055898b67dd3. It contains LICENSE, README.md, and DIKWP_NeuroBody_SemanticOS_Platform_Package.zip (351,172 bytes).

Evidence endpoint: <https://api.github.com/repos/YucongDuan/DIKWP-NeuroBody-SemanticOS-Platform-Package/git/trees/cc303795892906a331f7bad2d4f4055898b67dd3>

Response: a declared resource partition in the book-state runtime, a conservative resource-accounting exercise, and numerical units specified at each model boundary. No claim is made that the upstream archive lacks embodiment, resource models, or tests. The new resource model uses arbitrary units and is not a metabolism or immunity model.

4. C-NOESISCOPE — tree scope

Repository: <https://github.com/YucongDuan/C-NOESISCOPE>

Observed tree: 86103f5f480e1b63b4fe333d7d972821027cc7c9. It contains LICENSE, README.md, and DIKWP_MESH64C_NOESISCOPE_v1.0.0_CN_EN.zip (2,412,409 bytes).

Evidence endpoint: <https://api.github.com/repos/YucongDuan/C-NOESISCOPE/git/trees/86103f5f480e1b63b4fe333d7d972821027cc7c9>

Response: a transparent exercise with observationally matched dynamical realizations, an internal lesion, a report-channel lesion, sham handling, and restored parameters. This supports inspection of functional identifiability, not certification of consciousness. The internal algorithms or benchmarks of C-NOESISCOPE remain unaudited.

5. DIKWP-ProofLedger-OS — tree / README scope

Repository: <https://github.com/YucongDuan/DIKWP-ProofLedger-OS>

Observed tree: a9b7729a96de8dabc2a1f5230937d02062afe17c. It contains LICENSE, README.md, and dikwp_proofledger_os_open_source_app.zip (70,183 bytes).

Evidence endpoint: <https://api.github.com/repos/YucongDuan/DIKWP-ProofLedger-OS/git/trees/a9b7729a96de8dabc2a1f5230937d02062afe17c>

Its available description distinguishes verification scaffolding from a guarantee of truth. That boundary is retained, not presented as a newly discovered principle.

Response: hash-linked event records and separately retained checkpoints, with explicit tests for altered payloads, deletion in the middle, and suffix deletion when an expected head is supplied. These hashes establish consistency relative to a checkpoint; they do not establish empirical truth or prevent privileged full-log rewriting.

6. DIKWP-PACT-v0.1.0 — source-level scope

Repository: <https://github.com/YucongDuan/DIKWP-PACT-v0.1.0>

The inspected repository has expanded source, tests, schemas, benchmark materials, and a reproduction script. It must not be characterised as an archive-only concept. The README describes a

paired synthetic benchmark and explicitly limits what its structured signals demonstrate.

Two complete files were inspected:

`src/dikwp_pact/baselines.py`

Blob ID: `f523a8ab3fbed0fc49f76637128123f23afe36d7`.

Source: https://github.com/YucongDuan/DIKWP-PACT-v0.1.0/blob/main/src/dikwp_pact/baselines.py

`src/dikwp_pact/scoring.py`

Blob ID: `7ed83ee213f98df937793fa4d42b4b5da985000a`.

Source: https://github.com/YucongDuan/DIKWP-PACT-v0.1.0/blob/main/src/dikwp_pact/scoring.py

Source finding A: answer-dependent confidence

In `pact_reference`, the decision is selected from the provided categorical signals. The returned confidence is 0.94 when that decision equals the scenario's `expected_decision`, and 0.78 otherwise. Thus the reported confidence depends on an evaluator answer that would be unavailable during ordinary prediction.

This finding is narrowly about **confidence leakage**. The inspected decision rules do not use the expected answer to choose the decision, and the inspected aggregate scorer does not use this confidence field. Therefore it would be incorrect to claim that this line proves inflated decision accuracy or explains PACT's entire benchmark.

Implemented response: Observation contains only ID, two cues and their declared noise levels; Truth is a separate type. `predict()` rejects evaluator dictionaries. Tests flip all labels and verify that prediction probabilities do not change, while evaluation outcomes do. This is an API/implementation discipline, not a hostile-code sandbox: malicious Python code in the same process could still seek labels elsewhere.

Source finding B: conditional accuracy can hide missing cases

The inspected scorer marks missing traces, then builds a `valid` list containing only returned traces. Decision accuracy is computed over that list. Coverage is reported separately, but is not included in the displayed weighted composite. Consequently, a favourable returned subset can retain high conditional metrics while its coverage is low. Conditional accuracy is not intrinsically invalid; the problem is allowing it to stand in for complete-task performance without a missingness-sensitive measure.

Implemented response: `accuracy_all_cases` uses all declared truth cases as its denominator, so omissions are failures. `accuracy_observed_only` remains separately available and is null when no prediction is returned. Coverage and missing count are explicit. Missing binary Brier predictions receive loss 1. Duplicate prediction IDs, duplicate truth IDs and unknown returned IDs are rejected instead of silently merged. There is no unexplained overall score.

Source finding C: syntactic completeness is not semantic correctness

The inspected PACT scorer measures stage coverage from the presence of stage keys. Its residual metric checks whether a residual list is nonempty, and its recovery metric includes the presence of at least two recovery entries. Such measures can be useful structural checks; they cannot alone establish that a residual is honest, that a recovery is executable, or that evidence supports an assertion.

Implemented response: the companion does not claim to solve unrestricted semantic verification. It instead narrows each assurance to a behavior actually checked: dependency sets, exact revision links, rejected stale reads, resource conservation, consent transitions, numerical expectations and evidence-domain restrictions. Manual or externally parsed claims still require review.

Source finding D: preannotated signals constrain the conclusion

The PACT reference is a deterministic policy over supplied categories such as unauthorised access, purpose drift or corrupted evidence. Correct handling of those signals is valuable, but does not independently demonstrate that an agent can infer them from unconstrained language or real-world observations.

Implemented response: the cue laboratory computes probabilities from continuous synthetic observations before truth scoring, while the memory and runtime laboratories make evidence-state changes executable. However, these models also make strong assumptions: known noise, manually registered dependencies and declared owners. The new release does not convert those assumptions into a general intelligence claim.

What “advance” means in this delivery

The advance is the integration of the book's questions into an exposed, executable, regression-tested research workflow. The basic numerical methods, provenance ideas, and explicit authorisation principles are not claimed as newly invented algorithms. No head-to-head execution of all six upstream systems was possible. No statement of state-of-the-art accuracy, comprehensive portfolio superiority, independent adoption, or biological validation follows from this audit.